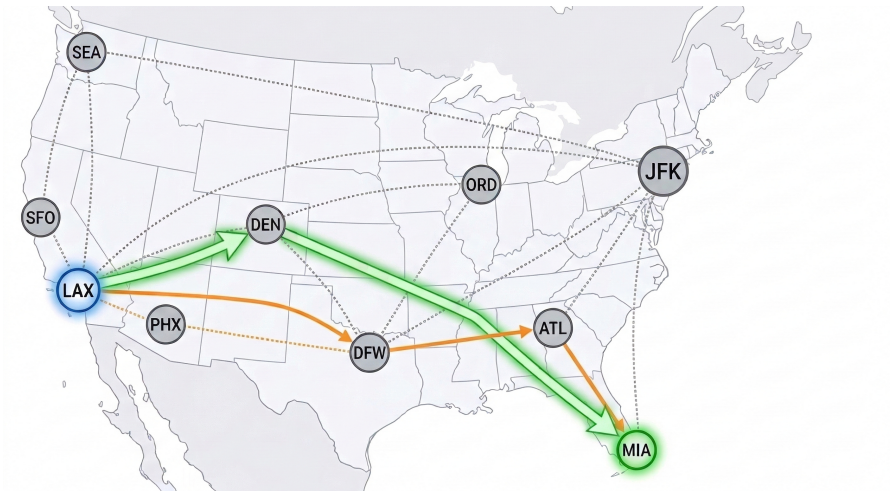


CS-GY 6033

Design and Analysis of Algorithms I

Lecture 7 | Part 1

Shortest Paths



Recall

- ▶ The **length** of a path is

$(\# \text{ of nodes}) - 1$

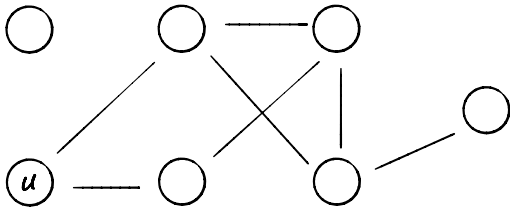
Definitions

- ▶ A **shortest path** between u and v is a path between u and v with smallest possible length.
 - ▶ There may be several, or none at all.
- ▶ The **shortest path distance** is the length of a shortest path.
 - ▶ Convention: ∞ if no path exists.
 - ▶ “the distance between u and v ” means spd.

Today: Shortest Paths

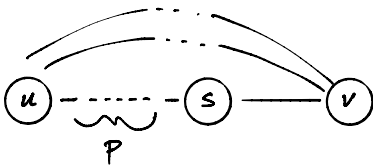
- ▶ **Given:** directed/undirected graph G , source u
- ▶ **Goal:** find shortest path from u to every other node

Example



Key Property

- ▶ A shortest path of length k is composed of:
 - ▶ A **shortest path** of length $k - 1$.
 - ▶ Plus one edge.



Algorithm Idea

- ▶ Find all nodes distance 1 from source.
- ▶ Use these to find all nodes distance 2 from source.
- ▶ Use these to find all nodes distance 3 from source.
- ▶ ...

It turns out...

...this is exactly what BFS does.

CS-GY 6033

Design and Analysis of Algorithms I

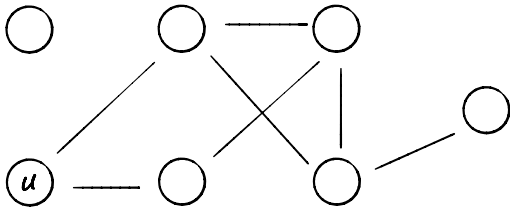
Lecture 7 | Part 2

BFS for Shortest Paths

Key Property of BFS

- ▶ For any $k \geq 1$ you choose:
- ▶ All nodes distance $k - 1$ from source are added to the queue before any node of distance k .
- ▶ Therefore, nodes are “processed” (popped from queue) in order of distance from source.

Example



Discovering Shortest Paths

- ▶ We “discover” shortest paths when we pop a node from queue and look at its neighbors.
- ▶ But the neighbor's status matters!

Consider This

- ▶ We pop a node s .
- ▶ It has a neighbor v whose status is **undiscovered**.
- ▶ We've discovered a **shortest path** to v through s !

Consider This

- ▶ We pop a node s .
- ▶ It has a neighbor v whose status is **pending** or **visited**.
- ▶ We already have a shortest path to v .

Modifying BFS

- ▶ Use BFS “framework”.
- ▶ Return dictionary of **search predecessors**.
 - ▶ If v is discovered while visiting u , we say that u is the **BFS predecessor** of v .
 - ▶ This encodes the shortest paths.
- ▶ Also return dictionary of shortest path distances.

```

def bfs_shortest_paths(graph, source):
    """Start a BFS at `source`."""
    status = {node: 'undiscovered' for node in graph.nodes}
    distance = {node: float('inf') for node in graph.nodes}
    predecessor = {node: None for node in graph.nodes}

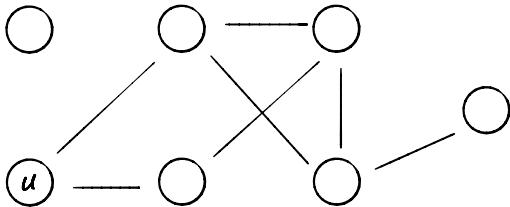
    status[source] = 'pending'
    distance[source] = 0
    pending = deque([source])

    # while there are still pending nodes
    while pending:
        u = pending.popleft()
        for v in graph.neighbors(u):
            # explore edge (u,v)
            if status[v] == 'undiscovered':
                status[v] = 'pending'
                distance[v] = distance[u] + 1
                predecessor[v] = u
                # append to right
                pending.append(v)
        status[u] = 'visited'

    return predecessor, distance

```

Example



CS-GY 6033
Design and Analysis of Algorithms I

Lecture 7 | Part 3

BFS Trees

Result of BFS

- ▶ Each node reachable from source has a single BFS predecessor.
 - ▶ Except for the source itself.
- ▶ The result is a **tree** (or forest).

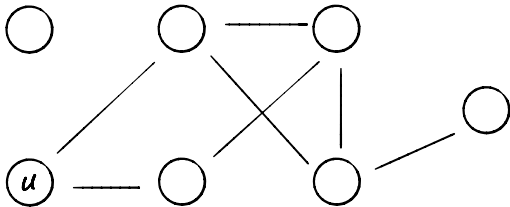
Trees

- ▶ A (free) **tree** is an undirected graph $T = (V, E)$ such that T is connected and $|E| = |V| - 1$.
- ▶ A **forest** is graph in which each connected component is a tree.

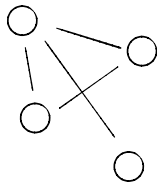
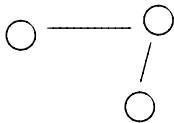
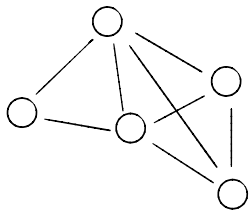
BFS Trees (Forests)

- ▶ If the input is connected, BFS produces a **tree**.
- ▶ If the input is not connected, BFS produces a **forest**.

Example



Example



BFS Trees

- ▶ BFS trees and forests encode shortest path distances.

CS-GY 6033

Design and Analysis of Algorithms I

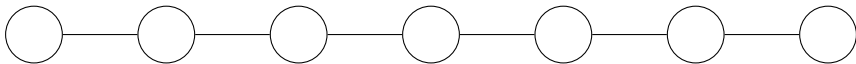
Lecture 7 | Part 4

Depth First Search

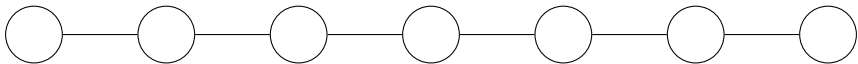
Visiting the Next Node

- ▶ Which node do we process next in a search?
- ▶ BFS: the **oldest** pending node.
- ▶ DFS (today): the **newest** pending node.
 - ▶ Naturally recursive.
 - ▶ Useful for solving different problems.

Example (BFS)



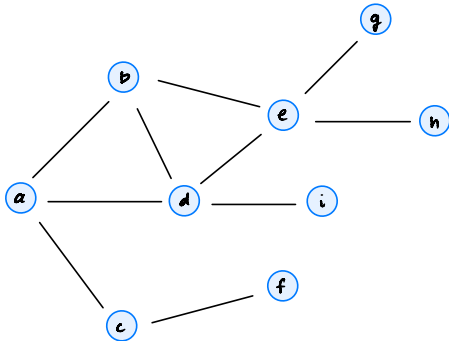
Example (DFS)



```
def dfs(graph, u, status=None):  
    """Start a DFS at `u`."""  
    # initialize status if it was not passed  
    if status is None:  
        status = {node: 'undiscovered' for node in graph.nodes}  
  
    status[u] = 'pending'  
    for v in graph.neighbors(u): # explore edge (u, v)  
        if status[v] == 'undiscovered':  
            dfs(graph, v, status)  
    status[u] = 'visited'
```

Exercise

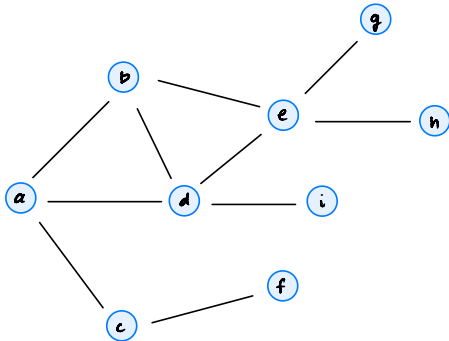
Write the nested function calls for a DFS on the graph below.



```
def dfs(graph, u, status=None):  
    """Start a DFS at `u`."""  
    ...  
    status[u] = 'pending'  
    for v in graph.neighbors(u): # explore edge (u, v)  
        if status[v] == 'undiscovered':  
            dfs(graph, v, status)  
    status[u] = 'visited'
```

Exercise

Write the nested function calls for a DFS on the graph below.



Differences

- ▶ In **BFS**, we “finish” a node u before moving on to the next.
- ▶ In **DFS**, we go to many other nodes, but “come back” to u .

Main Idea

We'll see that the nested structure of the **recursive function calls** gives us useful new information about the graph's structure.

Full DFS

- ▶ `dfs(u)` will visit all nodes **reachable** from u .
 - ▶ But not all nodes may be reachable from u !
- ▶ To visit **all** nodes in graph, need **full DFS**.

```
def full_dfs(graph):  
    status = {node: 'undiscovered' for node in graph.nodes}  
    for node in graph.nodes:  
        if status[node] == 'undiscovered'  
            dfs(graph, node, status)
```

```
def full_dfs(graph):
    status = {node: 'undiscovered' for node in graph.nodes}
    for node in graph.nodes:
        if status[node] == 'undiscovered':
            dfs(graph, node, status)

def dfs(graph, u, status=None):
    """Start a DFS at `u`."""
    # initialize status if it was not passed
    if status is None:
        status = {node: 'undiscovered' for node in graph.nodes}

    status[u] = 'pending'
    for v in graph.neighbors(u): # explore edge (u, v)
        if status[v] == 'undiscovered':
            dfs(graph, v, status)
    status[u] = 'visited'
```

Time Complexity

- ▶ In a full DFS:
 - ▶ dfs called on each node exactly once.
 - ▶ Like BFS, each edge is explored exactly:
 - ▶ once if directed
 - ▶ twice if undirected
- ▶ Time: $\Theta(V + E)$, **just like BFS.**

CS-GY 6033

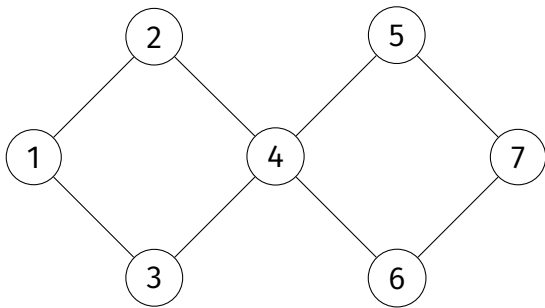
Design and Analysis of Algorithms I

Lecture 7 | Part 5

Nesting Properties of DFS

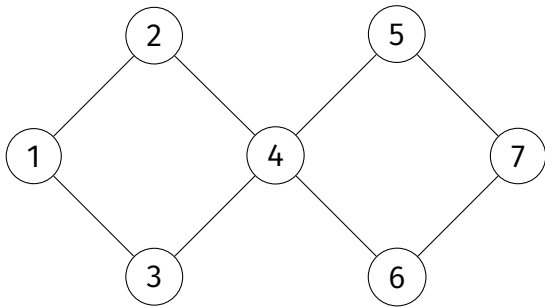
Exercise

True or False: if v is reachable from u and v is **undiscovered** when $\text{dfs}(u)$ is called, then $\text{dfs}(v)$ must be called during $\text{dfs}(u)$.



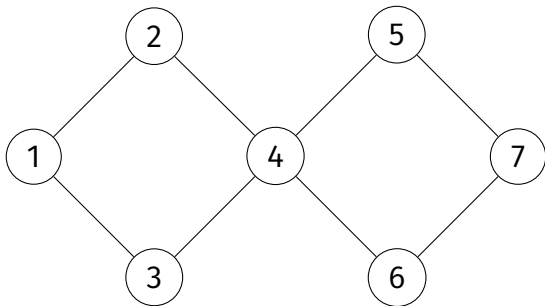
False!

- ▶ Suppose $\text{dfs}(4)$ is the root call.
 - ▶ When $\text{dfs}(1)$ is called, node 5 is undiscovered.
 - ▶ But $\text{dfs}(5)$ is **not** called during $\text{dfs}(1)$.



However..

- This intuition is correct if there is a path of **undiscovered** nodes from u to v when $\text{dfs}(u)$ is called.



Key Property of DFS (Informal)

- ▶ If at the time $\text{dfs}(u)$ is called...
 1. v is **undiscovered**; and
 2. there is a path of **undiscovered** nodes from u to v ,
- ▶ ...then $\text{dfs}(v)$ will **start and finish** during the call to $\text{dfs}(u)$.

Start and Finish Times

- ▶ Keep a running clock (an integer).
- ▶ For each node, record
 - ▶ **Start time**: time when marked **pending**
 - ▶ **Finish time**: time when marked **visited**
- ▶ Increment clock whenever node is marked **pending/visited**

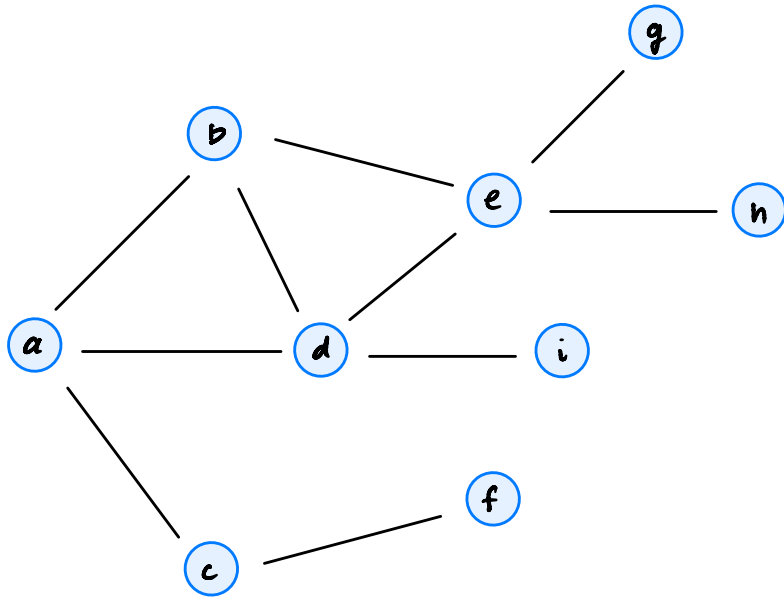
```
from dataclasses import dataclass

@dataclass
class Times:
    clock: int
    start: dict
    finish: dict

def full_dfs_times(graph):
    status = {node: 'undiscovered' for node in graph.nodes}
    predecessor = {node: None for node in graph.nodes}
    times = Times(clock=0, start={}, finish={})
    for u in graph.nodes:
        if status[u] == 'undiscovered':
            dfs_times(graph, u, status, times)
    return times, predecessor

def dfs_times(graph, u, status, predecessor, times):
    times.clock += 1
    times.start[u] = times.clock
    status[u] = 'pending'
    for v in graph.neighbors(u): # explore edge (u, v)
        if status[v] == 'undiscovered':
            predecessor[v] = u
            dfs_times(graph, v, status, times)
    status[u] = 'visited'
    times.clock += 1
    times.finish[u] = times.clock
```

Example



Key Property of DFS

- ▶ Suppose $\text{dfs}(u)$ is called before $\text{dfs}(v)$.
- ▶ If when $\text{dfs}(u)$ is called there is a path of **undiscovered** nodes from u to v , then:

$$\text{start}[u] < \text{start}[v] < \text{finish}[v] < \text{finish}[u].$$

- ▶ Otherwise:

$$\text{start}[u] < \text{finish}[u] < \text{start}[v] < \text{finish}[v].$$

Key Property

- ▶ Take any two nodes u and v ($u \neq v$).
- ▶ Assume for simplicity that $\text{start}[u] < \text{start}[v]$.
- ▶ Exactly one of these is true:
 - ▶ $\text{start}[u] < \text{start}[v] < \text{finish}[v] < \text{finish}[u]$
 - ▶ $\text{start}[u] < \text{finish}[u] < \text{start}[v] < \text{finish}[v]$

CS-GY 6033

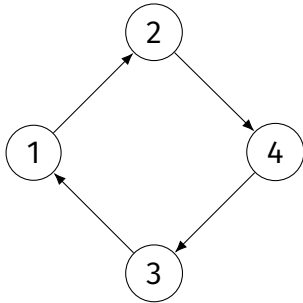
Design and Analysis of Algorithms I

Lecture 7 | Part 6

Cycles

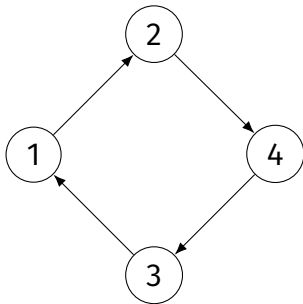
Cycle

- A **cycle** in a directed graph is a path that starts and ends at the same node.



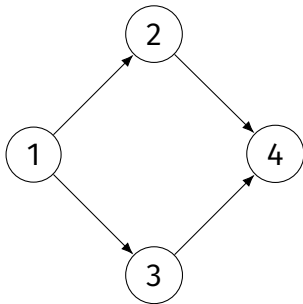
Cycle

- Alternatively: there is a **cycle** if u is reachable from v and v is reachable from u , for some $u \neq v$.



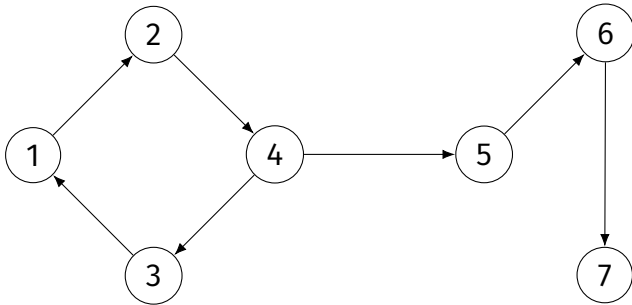
DAG

- ▶ A **directed acyclic graph** (DAG) is a directed graph with **no cycles**.



Cyclic Graphs

- ▶ A graph is cyclic even if it has only one cycle.
 - ▶ It doesn't have to be the whole graph.



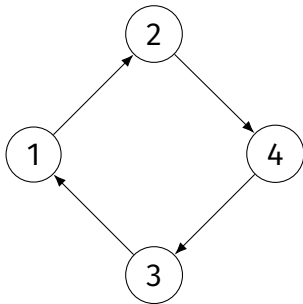
Detecting Cycles

- ▶ We check for cycles by looking for **back edges** in a full DFS.
- ▶ (u, v) is a **back edge** if while visiting node u , we see that v is **pending**.

```
...  
for v in graph.neighbors(u): # explore edge (u, v)  
    if status[v] == 'undiscovered':  
        dfs(graph, v, status)  
    elif status[v] == 'pending':  
        # back edge (u, v) found!  
...  

```


Example



Theorem

*A directed graph has a cycle **if (and only if)** a full DFS finds a back edge.*

Why?

- ▶ If a back edge (u, v) is found, then a cycle exists.
 - ▶ Suppose v is pending when we visit u .
 - ▶ This means that there is a path from v to u .
 - ▶ There is also a path from u to v .
 - ▶ So there is a cycle.

Why?

- ▶ If a cycle exists, then there is a back edge.
 - ▶ Suppose there is a cycle $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$.
 - ▶ Without loss of generality, assume v_1 is the first node in the cycle that is visited by the full DFS.
 - ▶ At the moment of $\text{dfs}(v_1)$, there is a path of undiscovered nodes between v_1 and v_k .
 - ▶ Therefore $\text{dfs}(v_k)$ will be called during $\text{dfs}(v_1)$.
 - ▶ During $\text{dfs}(v_k)$, we'll see the back edge.

CS-GY 6033

Design and Analysis of Algorithms I

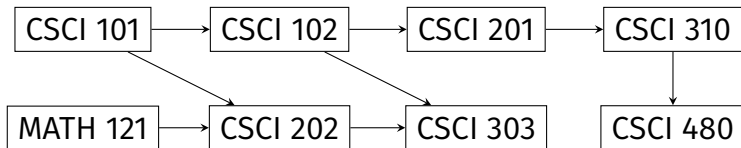
Lecture 7 | Part 7

Topological Sort

Applications of DFS

- ▶ Is node v reachable from node u ? **DFS**, **BFS**
- ▶ Is the graph connected? **DFS**, **BFS**
- ▶ How many connected components? **DFS**, **BFS**
- ▶ Find the shortest path between u and v . **DFS**, **BFS**
- ▶ Does the graph have a cycle? **DFS**, **BFS**

Prerequisite Graphs



Goal: find order in which classes should be taken in order to satisfy the prerequisites of CSCI 480.

Note

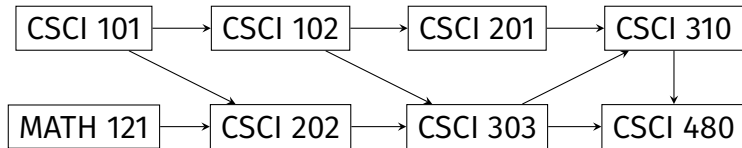
- ▶ Prerequisite graphs are¹ DAGs.

¹Or they should be, at least!

Topological Sorts

- ▶ **Given:** a DAG, $G = (V, E)$.
- ▶ **Compute:** an ordering of V such that if $(u, v) \in E$, then u comes before v in the ordering
- ▶ This is called a **topological sort** of G .

Example



MATH 121, CSCI 101, CSCI 202, CSCI 102, CSCI 303, CSCI 201, CSCI 310, CSCI 480

Computing a Topological Sort

- ▶ How do we compute a topological sort, algorithmically?
- ▶ **Observation:** if v is reachable from u , v **must** come **after** u in the topological sort.



Exercise

Suppose v is reachable from u in a DAG.

True or false: after a full DFS, $\text{finish}[v] < \text{finish}[u]$.

Claim

- If v is reachable from u in a DAG, then:

$$\text{finish}[v] < \text{finish}[u]$$

Idea

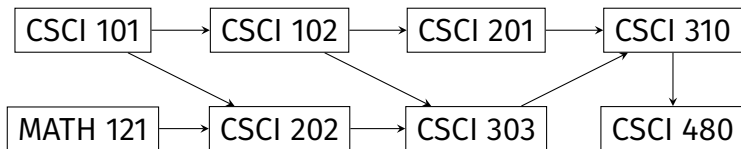
- ▶ Take any two nodes u and v ($u \neq v$).
- ▶ Assume the graph is a DAG, run DFS.
- ▶ If v is reachable from u , then $\text{finish}[v] < \text{finish}[u]$.

Putting it together...

- ▶ **Observation:** If v is reachable from u , then v must come after u in the topological sort.
- ▶ **Recall:** If v is reachable from u , then $\text{finish}[v] < \text{finish}[u]$.

Exercise

Compute start and finish times using CSCI 101 as the source.



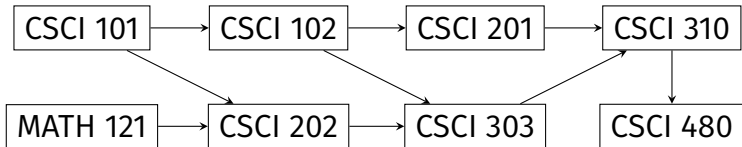
Idea

- ▶ **Observation:** If v is reachable from u , then v must come after u in the topological sort.
- ▶ **Recall:** If v is reachable from u , then $\text{finish}[v] < \text{finish}[u]$.
- ▶ **Therefore:** if $\text{finish}[v] < \text{finish}[u]$, then v must come after u in the topological sort.
- ▶ **Idea:** sort nodes in **descending** order by finish time.

Algorithm

- ▶ To find a topological sort (if it exists):
 - ▶ Compute times with Full DFS.
 - ▶ Sort in **descending** order by finish time.
- ▶ Time complexity:

Example



Note

- ▶ There can be many valid topological sorts!

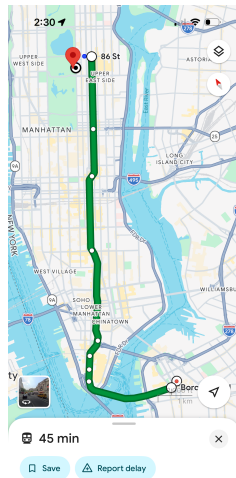
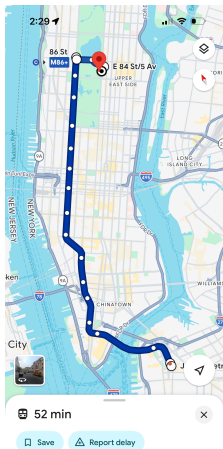
CS-GY 6033

Design and Analysis of Algorithms I

Lecture 7 | Part 8

Shortest Paths in Weighted Graphs

Google Maps



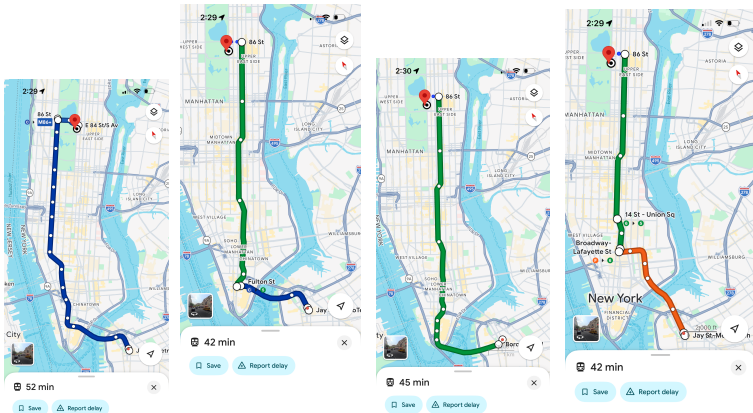
Weighted Graphs

An **edge weighted graph** $G = (V, E, \omega)$ is a triple where (V, E) is a graph and $\omega : E \rightarrow \mathbb{R}$ maps each edge to a **weight**.

- ▶ Can be directed or undirected.
- ▶ In general, weights can be positive, negative, zero.
- ▶ Many uses, such as representing **metric spaces**.

Path Lengths

The **length** of a path in a **weighted graph** (usually) refers to the total weight of all edges in the path.

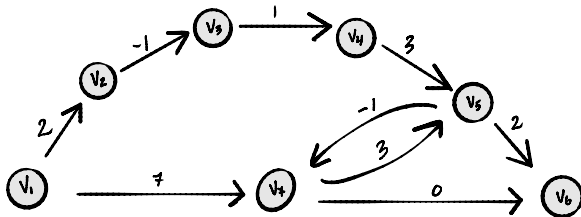


Shortest Paths

- ▶ A **shortest path** between u and v is a path between u and v with minimum length.
 - ▶ In other words, minimum total weight.

Example

What is the shortest path from v_1 to v_6 ?



Path:

Length:

Today (and next time)

How do we find shortest paths in weighted graphs?

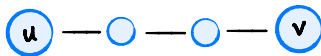
Idea #0

- ▶ Does BFS work?
 - ▶ **No**, not really. Only if all weights are the same.
- ▶ Can we “convert” a weighted graph to an unweighted one?

Idea #0



Idea #0



Idea #0

- ▶ Step 1: “Convert” weighted graph to unweighted one with dummy nodes.
- ▶ Step 2: Call BFS on this new graph.

Idea #0

- ▶ **Very inefficient** for large weights.
- ▶ What if edge weights are floats, or negative?

Ideas #1 and #2

- ▶ We'll look at two algorithms: **Bellman-Ford** and **Dijkstra's**.
INPUT: weighted graph, source vertex s .
OUTPUT: shortest paths from s to every other node.
- ▶ Both work by:
 - ▶ keeping track of shortest known path (**estimates**).
 - ▶ iteratively **updating** these until they're correct.

Shortest Path Estimates

- ▶ B-F and Dijkstra's keep track of the shortest paths found so *far*; we call these the **estimated shortest paths**.
- ▶ For each node u , remember u 's:
 - ▶ predecessor in estimated shortest path;
 - ▶ distance from source s in estimated shortest path.
- ▶ **Key:** estimated distance will always be $\geq$ actual distance.

Updates

- ▶ Both algorithms work by iteratively updating their estimates.
- ▶ On each iteration, consider a new edge (u, v) . Ask: is the best known shortest path from

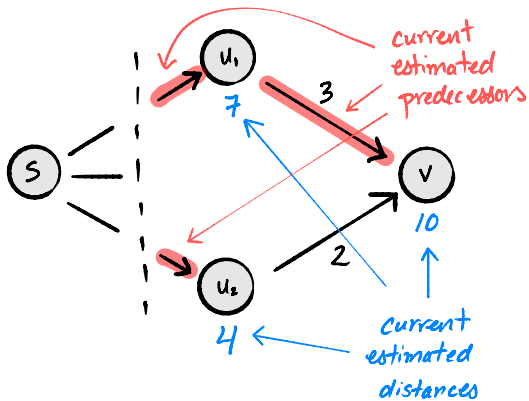
$\text{source} \rightarrow \dots \rightarrow u \rightarrow v$

shorter than the best known shortest path from

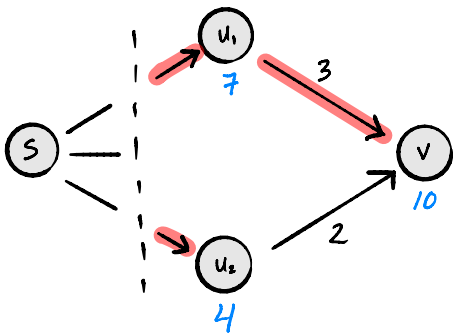
$\text{source} \rightarrow \dots \rightarrow \text{predecessor}[v] \rightarrow v?$

- ▶ If it is, we have discovered a shorter path to v .

Example: Updating (u_2, v) :

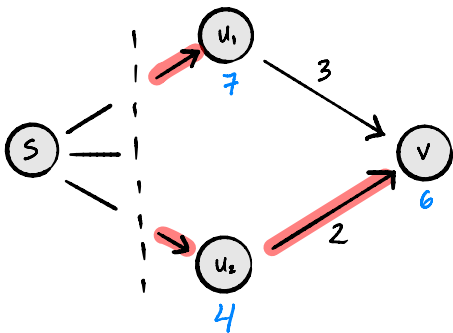


Example: Updating (u_2, v) :



estimated length of $s \rightarrow u_2 \rightarrow v$
= (estimated length of $s \rightarrow u_2$) + $w(u_2, v)$
= $4 + 2 = 6 < 10$

Example: After Updating (u_2, v):



A shorter path has been found.

Updating, in Code

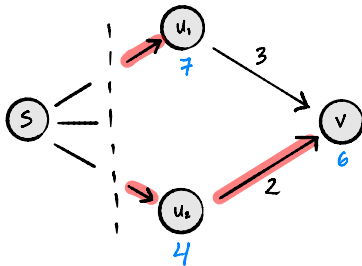
- ▶ Let:
 - ▶ `est` be a dictionary of estimated shortest path distances.
 - ▶ `predecessor` be a dictionary of estimated shortest path predecessors.
 - ▶ `weights` be a function which returns edge weights.

Updating, in Code

```
def update(u, v, weights, est, predecessor):  
    """Update edge (u,v)."""  
    if est[v] > est[u] + weights(u,v):  
        est[v] = est[u] + weights(u,v)  
        predecessor[v] = u  
        return True  
    else:  
        return False
```

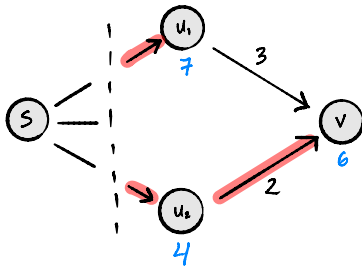
► Time complexity: _____

When does an update discover a shortest path?



- Suppose updating (u_2, v) finds a shorter path to v .
- True or False: the actual shortest path must go through u_2 .

When does an update discover a shortest path?



- ▶ Suppose updating (u_2, v) finds a shorter path to v .
- ▶ True or False: the actual shortest path must go through u_2 .
- ▶ **False:** we might later discover a better path to u_1 .

When does an update discover the shortest path?

- ▶ Let (u, v) be an edge.
- ▶ Suppose:
 - ▶ the actual shortest path to u has been found;
 - ▶ the actual shortest path to v goes through (u, v) .
- ▶ Then after updating (u, v) , the estimated shortest path to v is correct.

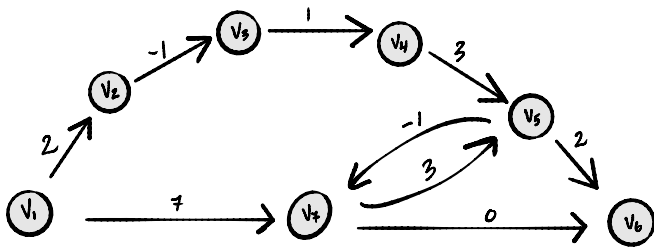
CS-GY 6033
Design and Analysis of Algorithms I

Lecture 7 | Part 9

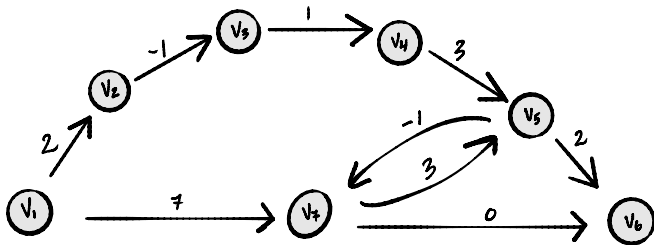
Bellman-Ford

Intuition

- ▶ Shortest paths that have many edges are “harder” to discover.
 - ▶ May require many updates.
- ▶ Shortest paths that have few edges are “easier” to discover.
- ▶ Once we've discovered all of the shortest paths with few edges, it makes it easier to discover the shortest paths with more edges.

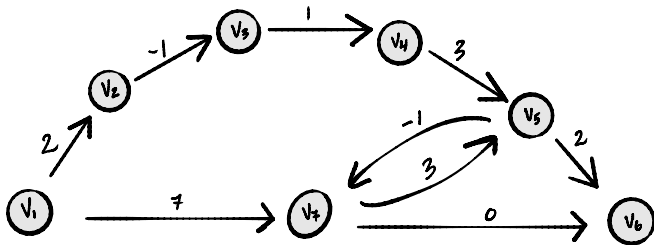


Updating All Edges



- Suppose we update all of the edges, one by one.
- Then all nodes whose shortest path from s has only **one** edge are **guaranteed** to be estimated correctly.

Updating All Edges



- Suppose we update all of the edges again.
- Then all nodes whose shortest path from s has at most **two** edges are **guaranteed** to be estimated correctly.

Loop Invariant

- ▶ One iteration: update all edges in arbitrary order.
- ▶ Loop invariant: After α iterations, all nodes whose shortest path from s has $\leq \alpha$ edges are guaranteed to be estimated correctly.

The Bellman-Ford Algorithm

```
def bellman_ford(graph, weights, source):  
    """Assume graph is directed."""  
    est = {node: float('inf') for node in graph.nodes}  
    est[source] = 0  
    predecessor = {node: None for node in graph.nodes}  
  
    for i in range(?):  
        for (u, v) in graph.edges:  
            update(u, v, weights, est, predecessor)  
  
    return est, predecessor
```

Bellman-Ford

- ▶ Claim: each node must have a shortest path which is simple².
- ▶ The most edges a simple path can have is $|V| - 1$
- ▶ Idea of Bellman-Ford: iteratively update all edges, repeat $|V| - 1$ times.

²Edge case: cycles of weight zero.

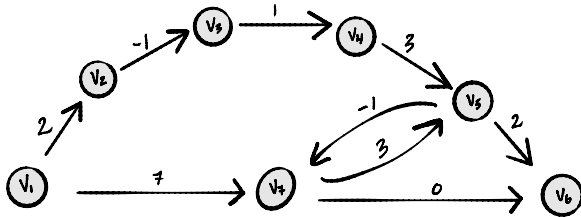
The Bellman-Ford Algorithm

```
def bellman_ford(graph, weights, source):  
    """Assume graph is directed."""  
    est = {node: float('inf') for node in graph.nodes}  
    est[source] = 0  
    predecessor = {node: None for node in graph.nodes}  
  
    for i in range(len(graph.nodes) - 1):  
        for (u, v) in graph.edges:  
            update(u, v, weights, est, predecessor)  
  
    return est, predecessor
```

Example

Suppose `graph.edges` returns edges in following order:

$(v_3, v_4), (v_1, v_2), (v_2, v_3), (v_7, v_6), (v_5, v_7),$
 $(v_7, v_5), (v_4, v_5), (v_5, v_6), (v_1, v_7)$



Time Complexity

```
def bellman_ford(graph, weights, source):  
    """Assume graph is directed."""  
    est = {node: float('inf') for node in graph.nodes}  
    est[source] = 0  
    predecessor = {node: None for node in graph.nodes}  
  
    for i in range(len(graph.nodes) - 1):  
        for (u, v) in graph.edges:  
            update(u, v, weights, est, predecessor)  
  
    return est, predecessor
```

- ▶ Setup: _____ time
- ▶ Each update takes _____ time
- ▶ There are exactly _____ updates
- ▶ Total time complexity: _____

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 7 | Part 10

Early Stopping and Negative Cycles

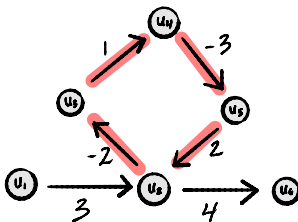
Early Stopping

- ▶ B-F may not need to run for $|V| - 1$ iterations.
- ▶ If no predecessors change, we can break:

```
def bellman_ford(graph, weights, source):  
    """Early stopping version."""  
    est = {node: float('inf') for node in graph.nodes}  
    est[source] = 0  
    predecessor = {node: None for node in graph.nodes}  
  
    for i in range(len(graph.nodes) - 1):  
        any_changes = False  
        for (u, v) in graph.edges:  
            changed = update(u, v, weights, est, predecessor)  
            any_changes = changed or any_changes  
        if not any_changes:  
            break  
    return est, predecessor
```


Negative Cycles

- A **negative cycle** is a cycle whose total edge weight is negative:



- If a graph has a negative cycle, (some) shortest paths are **not well defined**.

Detecting Negative Cycles

- ▶ If graph **does not have** negative cycles, estimated distances eventually stop changing (after at most $|V| - 1$ iterations).
- ▶ If graph **has** negative cycles, estimated distances always decrease.
- ▶ To detect them: run a $|V|$ th iteration; if distances change, a negative cycle exists.

Detecting Negative Cycles

```
def bellman_ford(graph, weights, source):  
    """Early stopping version, detects negative cycles."""  
    est = {node: float('inf') for node in graph.nodes}  
    est[source] = 0  
    predecessor = {node: None for node in graph.nodes}  
  
    for i in range(len(graph.nodes)):  
        any_changes = False  
        for (u, v) in graph.edges:  
            changed = update(u, v, weights, est, predecessor)  
            any_changes = changed or any_changes  
        if not any_changes:  
            break  
    # this will be True if negative cycles exist  
    contains_negative_cycles = any_changes  
    return est, predecessor, contains_negative_cycles
```